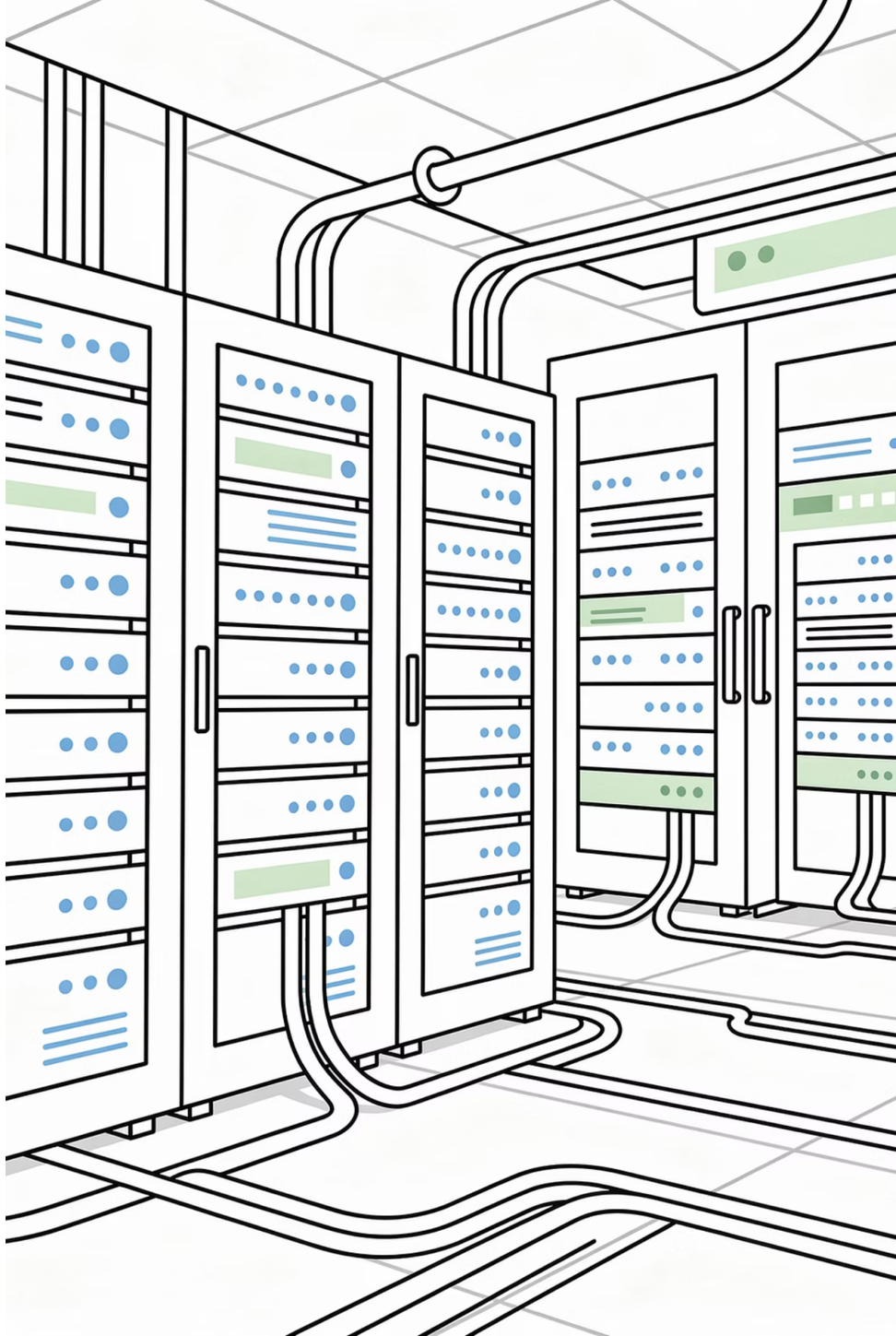
Өгөгдлийн сангийн удирдлага

F.ITM237

Мэдээллийн технологийн тэнхим

С.Өлзийбаяр (Ph.D)





SQL үндэс суурь: Уламжлалаас орчин үеийн эрин үе хүртэлх аялал

МТ болон КУ-ны оюутнуудад зориулсан мэдээллийн сангийн менежментийн цогц ойлголтын хүрээнд SQL-ийн үндэс суурь, DDL болон DML командууд, уламжлалт холбоост өгөгдлийн сангаас орчин үеийн NoSQL орчин руу шилжих явцыг хамарсан онол, практикий өргөн хүрээтэй ойлголт өгнө.

ТАНИЛЦУУЛГА

Хичээлийн тойм ба суралцах зорилтууд

Үндсэн ойлголт

SQL-ийн үндэс суурь синтакс, DDL (Өгөгдлийн тодорхойлолтын хэл) болон DML (Өгөгдлийн удирдлагын хэл) командуудыг эзэмших. Мэдээллийн сангийн архитектурын үндсийг ойлгох.

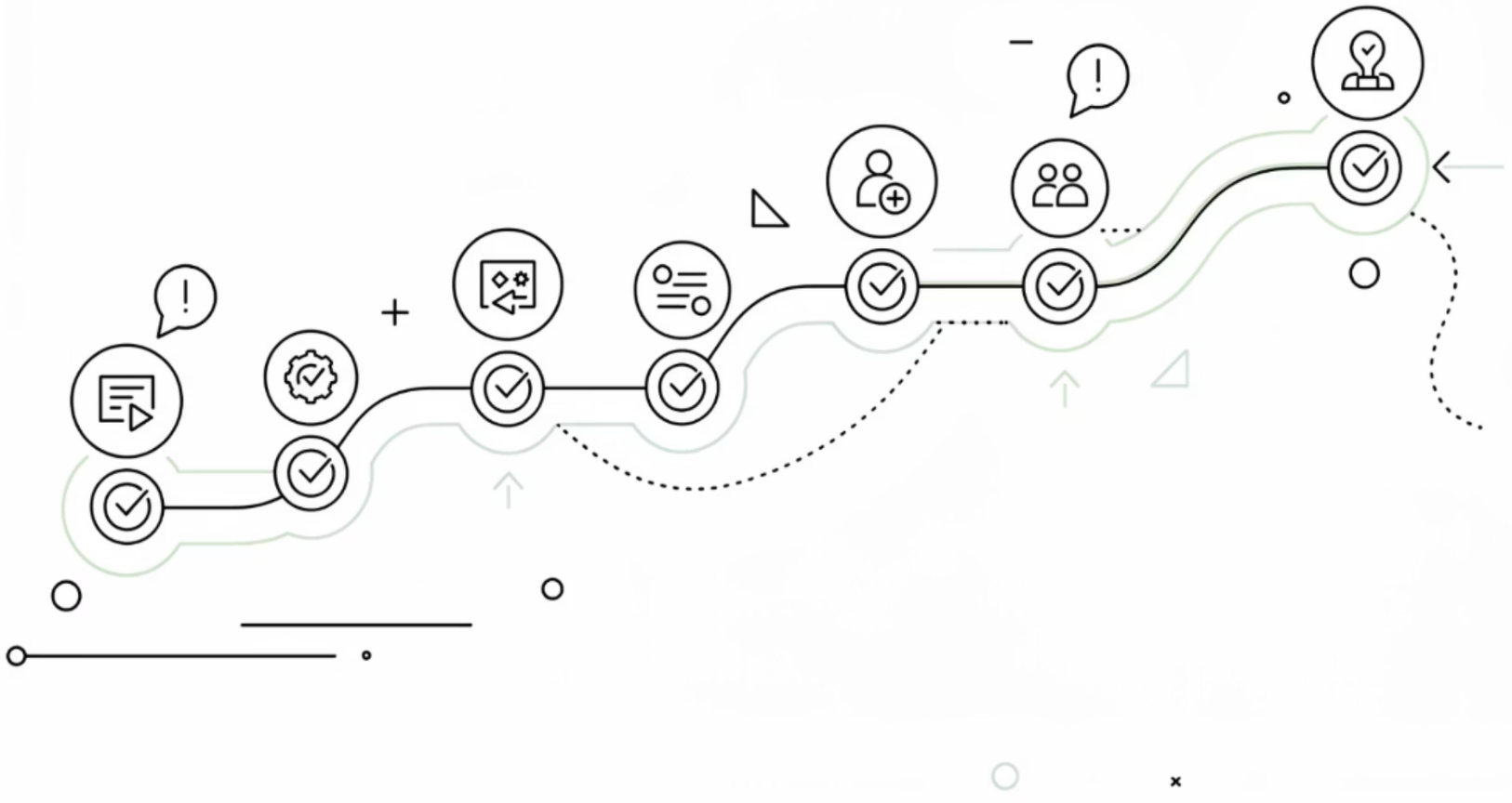
Практик хэрэглээ

SELECT, WHERE, ORDER BY ашиглан үр дүнтэй асуулга бичих чадвар эзэмших. Асуулгын оновчлолын арга техникийг ойлгож практикт хэрэглэх.

Орчин үеийн контекст

SQL хэрхэн уламжлалт харилцааны мэдээллийн сан болон орчин үеийн NoSQL орчинд ажилладагийг судлах. Технологийн хувьслыг ойлгох.

Бидний өнөөдөр хамрах сэдвүүд



01

SQL танилцуулга

Түүх, ач холбогдол, синтаксын үндэс

02

DDL командууд

Өгөгдлийн сангийн бүтэц үүсгэх, өөрчлөх

03

DML үйлдлүүд

Өгөгдлийг үр дүнтэй удирдах, боловсруулах

04

Асуулгын үндэс

Үр дүнтэй SELECT мэдэгдэл бичих

05

Уламжлал ба орчин үе

Өгөгдлийн сангийн хандлагын хувьсал

06

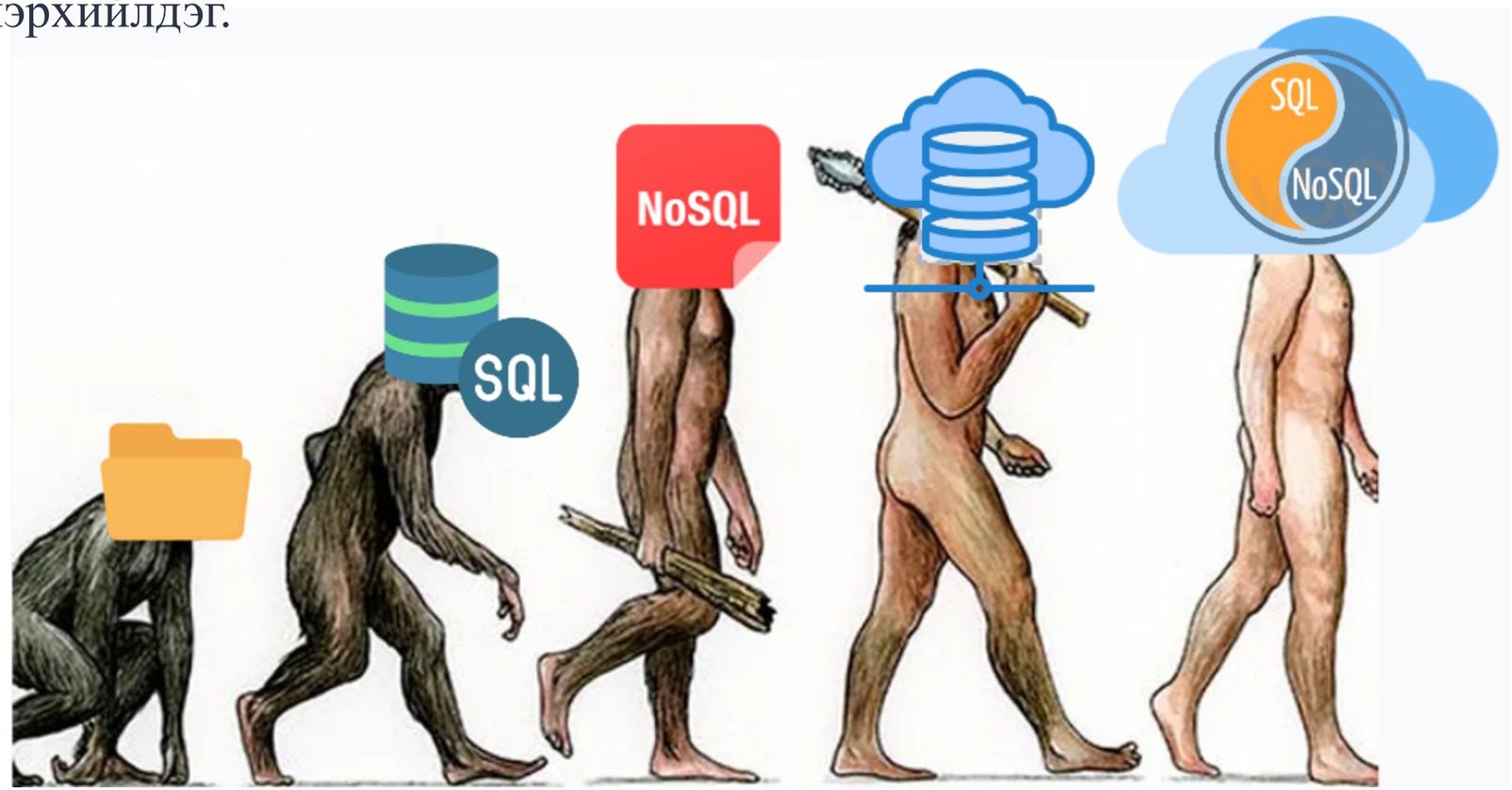
Практик дасгал

Бодит SQL асуулга бичих туршлага

БҮЛЭГ 1

SQL-ийн хувьсал: Түүхэн хэтийн төлөв

- SQL-ийн аялал таван арван жилийг хамарч, IBM-ийн судалгааны төслөөс эхлээд өгөгдөл удирдах хэлний дэлхийн стандарт болж хувирсан түүхтэй. Энэхүү хувьсал нь зөвхөн технологийн ахиц дэвшил биш, харин мэдээллийг хадгалах, сэргээн авах талаарх бидний сэтгэлгээний үндсэн өөрчлөлтийг илэрхийлдэг.



SQL-ийн төрөлт ба эрт үеийн хөгжил



1

1970: Харилцааны загвар

Кодд IBM-д харилцааны загварыг тодорхойлсон

2

1974: SEQUEL

IBM SEQUEL хэл боловсруулсан

3

1979: Анхны арилжааны бүтээгдэхүүн

Oracle SQL өгөгдлийн санг зах зээлд гаргасан

4

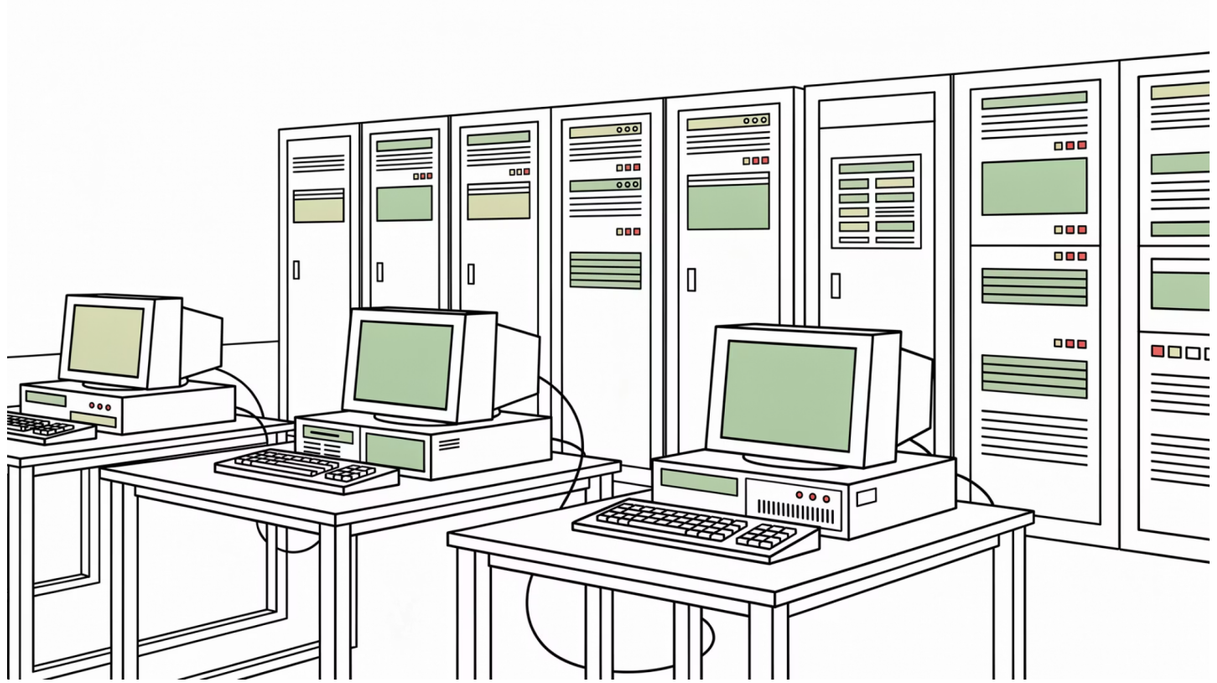
1986: ANSI стандарт

SQL ANSI-ээр стандартчлагдсан

Уламжлалт SQL эрин үе: 1980–2000

Тодорхойлох шинж чанарууд

Уламжлалт SQL эрин үе нь хатуу схем, хүчтэй ACID гүйлгээний баталгаа, болон ***яаж*** биш харин ***юуг*** авахад анхаардаг declarative хандлагаар тодорхойлогддог байсан. Аж ахуйн нэгжийн системүүд энэхүү тогтвортой байдал, таамаглах чадварт ихээхэн найдаж байв.



- *Хатуу схемийн хяналт нь өгөгдлийн бүрэн бүтэн байдлыг хангасан*
- *ACID шинж чанарууд нь гүйлгээний найдвартай байдлыг өгсөн*
- *Declarative синтакс нь асуулга бичихийг хялбаршуулсан*
- *Аж ахуйн нэгжийн програмын архитектурт давамгайлсан*

Орчин үеийн SQL хувьсгал: 2000-өнөөг хүртэл

2009: NoSQL гарч ирэх

MongoDB болон Cassandra уламжлалт загваруудыг уян хатан схем, хэвтээ масштабжуулалтаар сорьсон

2012: NewSQL үүсэх

Google Spanner болон CockroachDB SQL-ийг тархсан архитектуртай хослуулсан

2015: Үүлэн суурьтай өгөгдлийн сан

AWS Aurora болон Azure SQL уян масштабжилт, удирдлагатай үйлчилгээнүүдийг авчирсан

2020: SQL+JSON нэгдэл

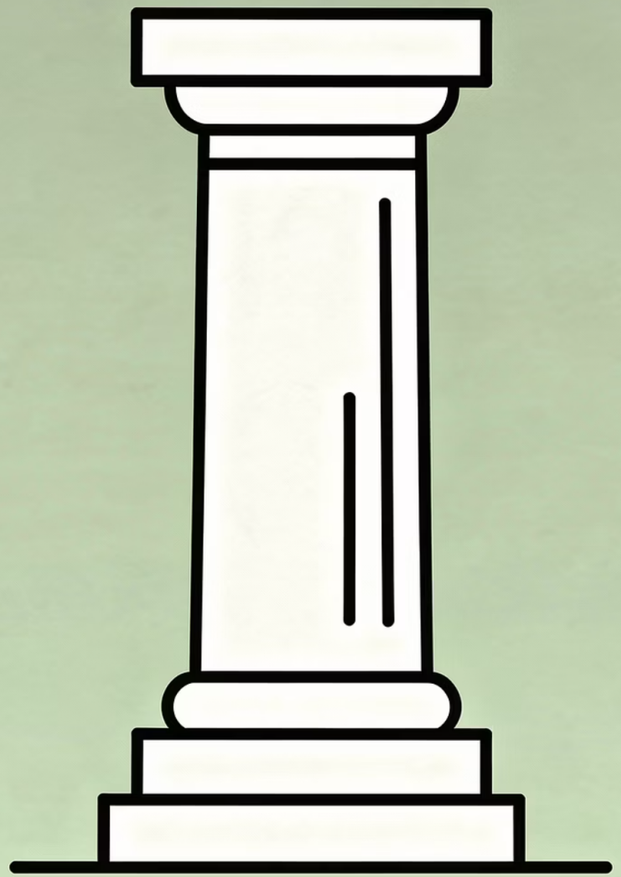
PostgreSQL JSONB болон бусад эрлийз загварууд уян хатан байдлыг бүтэцтэй хослуулсан

Өнөөгийн өгөгдлийн сангийн системүүд дэх SQL дэмжлэг

NoSQL-ийн өсөлтийг үл харгалзан SQL гайхалтай уян хатан байсаар байна. Орчин үеийн өгөгдлийн сангийн системүүдийн ихэнх нь ямар нэгэн хэлбэрээр SQL дэмждэг бөгөөд энэ нь түүний өгөгдлийн хэл болох олон улсын үнэ цэнийг харуулж байна. Хүснэгт нь өөр өөр төрлийн өгөгдлийн сангууд SQL-ийг хэрхэн хүлээн авсныг үзүүлж байна.

Өгөгдлийн сангийн төрөл	Жишээ	SQL дэмжлэг
Уламжлалт RDBMS	PostgreSQL, MySQL	100% стандарт
Үүлэн SQL	AWS RDS, Azure SQL	100% + өргөтгөлүүд
NewSQL	CockroachDB	95% нийцтэй
NoSQL	MongoDB	50-70% SQL-тэй төстэй
Шинжилгээний	BigQuery, Snowflake	100% + шинжилгээ

Энэхүү өргөн хүрээний хэрэглээ нь SQL-ийн declarative загвар, стандартчлагдсан синтакс нь доод хадгалалтын архитектураас үл хамааран өгөгдөлтэй харилцахад өвөрмөц үнэ цэнийг өгсөөр байгааг нотолж байна.



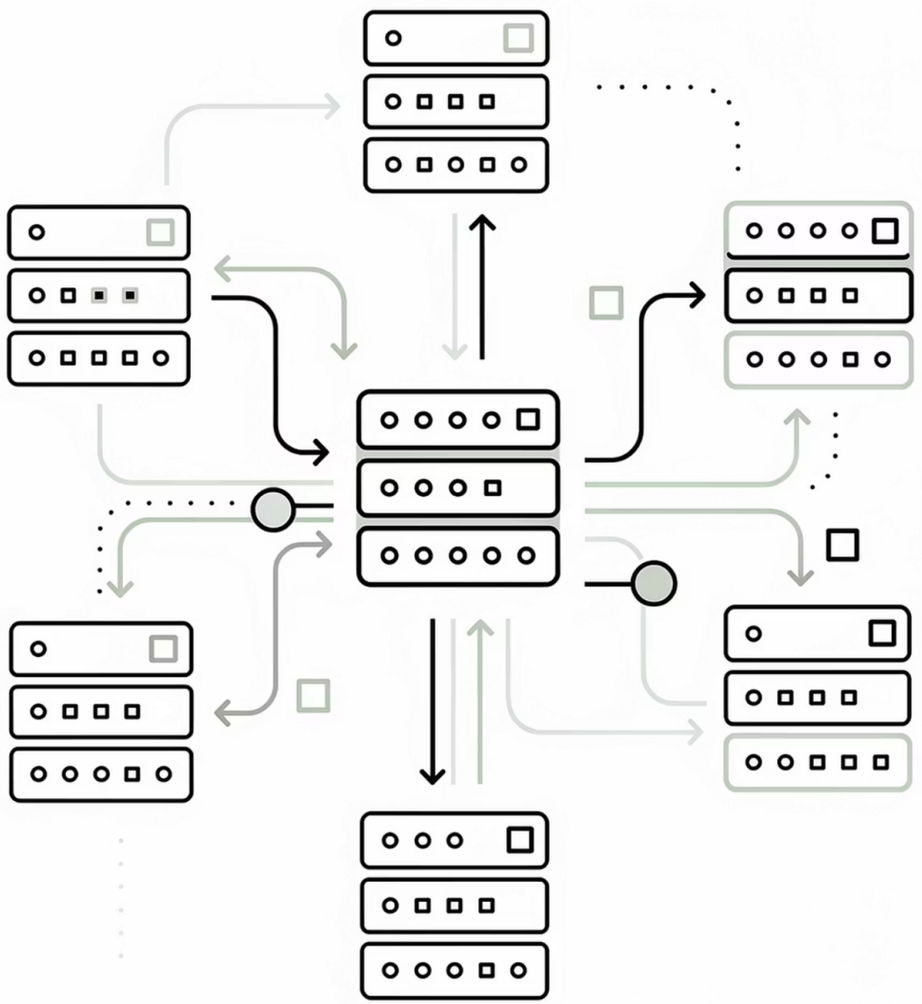
Гол ойлголт: SQL–ийн мөнхийн хүч

50+ жил

Яагаад SQL амьд үлдсэн

- NoSQL системүүд масштабжилтын сорилтыг шийдвэрлэхээр гарч ирсэн ч эцэст нь SQL-тэй төстэй интерфейсүүдийг нэмсэн. Энэ нь буцаж унах биш — хүлээн зөвшөөрөх явдал юм. SQL-ийн declarative шинж чанар, стандартчлал, хүнд ойлгомжтой байдал нь өгөгдөлтэй харилцахад орлуулашгүй болгодог.

- SQL хойшоо нийцтэй байдлаа хадгалсан байхад тасралтгүй хувирсан. Хэлний уян хатан байдал, хүч нь шинэ парадигмуудад дасан зохицохыг боломжтой болгосон бөгөөд үндсэн давуу талуудаа хадгалсан хэвээр байна. Хатуу схемээс эхлээд уян хатан JSON баримтууд хүртэл, SQL өөрийн онцлогийг алдалгүй хувирдаг.



БҮЛЭГ 2

SQL үндэс ойлголтыг ойлгох

SQL (Structured Query Language) нь харилцааны өгөгдлийн сантай харилцах стандарт хэл юм. Түүний declarative шинж чанар нь та ***юу*** өгөгдөл хүсч байгаагаа заах боловч ***хэрхэн*** авахыг дэлгэрэнгүй зааж өгөхгүй байхыг зөвшөөрдөг — өгөгдлийн сангийн хөдөлгүүр оновчлолыг автоматаар гүйцэтгэдэг.

Declarative болон Imperative: Үндсэн ялгаа

Imperative хандлага (JavaScript)

```
let result = [];
for (let i = 0; i < students.length; i++) {
  if (students[i].age > 20) {
    result.push({
      name: students[i].name,
      age: students[i].age
    });
  }
}
```

Та **алхам бүрийг** зааж өгнө: массиваар давталт хийх, нөхцөл шалгах, үр дүн бүтээх.

Declarative хандлага (SQL)

```
SELECT name, age
FROM students
WHERE age > 20;
```

Та **зөвхөн үр дүнг** заадаг: ямар өгөгдөл хүсч байгаа. Өгөгдлийн сан индекс, хүснэгт сканнер эсвэл бусад оновчлолын стратегиудыг ашиглахаа шийднэ.

SQL-ийн үндсэн шинж чанарууд

Стандартчлагдсан хэл

1986 оноос хойш ANSI/ISO стандарт, өгөгдлийн сангийн системүүдийн хооронд зөөвөрлөх чадварыг хангана

Олон зориулалтын

Бүтцийн тодорхойлолт, өгөгдөл боловсруулалт, хандалтын хяналт, гүйлгээг боловсруулдаг

Дэлхийн хэрэглээ

MySQL-ээс BigQuery хүртэл бараг бүх орчин үеийн өгөгдлийн сангийн системд дэмжигддэг

SQL синтаксын конвенци ба шилдэг практикууд

Чухал дүрмүүд

- **Case-insensitive түлхүүр үгс:** SELECT нь select-тэй тэнцүү боловч конвенцээр түлхүүр үгсийг том үсгээр бичнэ
- **Цэгтэй таслалын төгсгөл:** Мэдэгдлүүд цэгтэй таслалаар төгсдөг
- **Тайлбарууд:** Нэг мөрт -- ашиглах, олон мөрт /* */ ашиглах
- **Мөр таслалтууд:** Нарийн асуулгыг уншигдахуйц хэсэг болгон таслах
- **Нэрлэх конвенцүүд:** snake_case эсвэл camelCase-ийг тогтмол ашиглах



```
-- Сайн форматчлал
SELECT student_id, name, department
FROM students
WHERE age > 20
ORDER BY name;
```

SQL мэдэгдлийн ангиллууд

DDL: **Өгөгдлийн тодорхойлолт**

CREATE, ALTER, DROP, TRUNCATE — Өгөгдлийн сангийн бүтэц, схемийг тодорхойлох, өөрчлөх

DML: **Өгөгдөл боловсруулалт**

SELECT, INSERT, UPDATE, DELETE — Хүснэгт доторх өгөгдөлд асуулга тавих, өөрчлөх

DCL: **Өгөгдөл хяналт**

GRANT, REVOKE — Хэрэглэгчийн зөвшөөрөл, хандах эрхийг удирдах

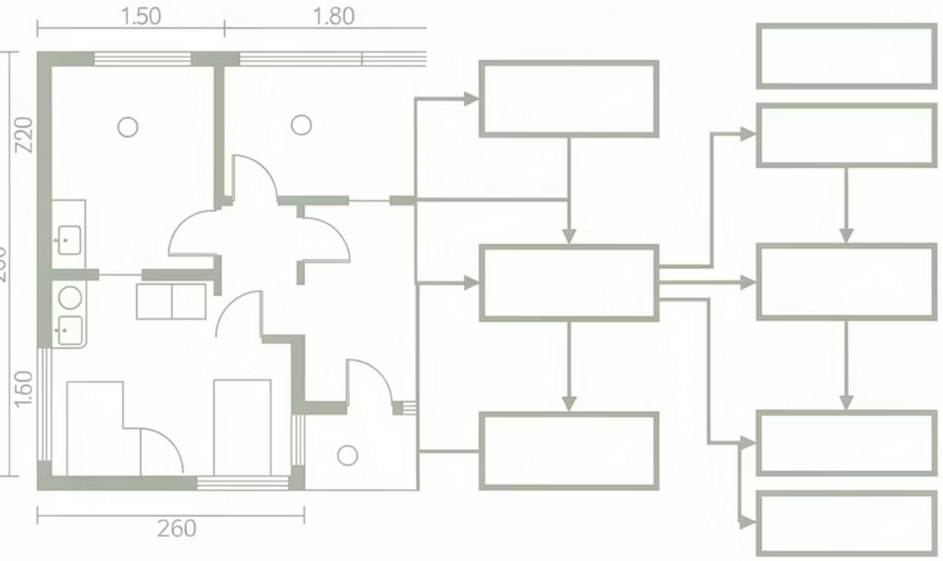
TCL: **Гүйлгээ хяналт**

BEGIN, COMMIT, ROLLBACK — Гүйлгээ боловсруулалт, өгөгдлийн тогтвортой байдлыг хянах



Яагаад SQL өгөгдлийн сангийн менежментэд чухал вэ

- SQL нь зөвхөн өгөгдөл боловсруулах хэрэгсэл биш — энэ бол аж ахуйн нэгжийн өгөгдөл удирдах үндэс юм. Стандартчлагдсан, declarative шинж чанараараа SQL нь програм хангамжийн инженерүүдээс өгөгдлийн шинжээч, бизнесийн шинжээчид хүртэл олон мэргэжилтнүүдэд өгөгдөлтэй харилцах боломжийг олгодог.
- Орчин үеийн өгөгдлийн сангийн систем бүр SQL эсвэл SQL-тэй төстэй интерфейстэй байдаг нь түүний өвөрмөц үнэ цэнийг нотолж байна. Та NoSQL, NewSQL эсвэл уламжлалт RDBMS ашиглаж байгаа эсэхээс үл хамааран SQL ур чадвар нь өгөгдлийн мэргэжилтэнд зайлшгүй шаардлагатай.



БҮЛЭГ 3

DDL: Өгөгдлийн тодорхойлолтын хэл

DDL командууд нь өгөгдлийн сангийн архитектурын суурийг бүрдүүлдэг. Тэд таны өгөгдлийг байршуулах бүтцийг бүтээх, өөрчлөх, устгах — хүснэгтүүд, индексүүд, өгөгдлийн сангууд болон өгөгдлийн бүрэн бүтэн байдлыг хангах хязгаарлалтууд.

CREATE DATABASE: Суурь ойлголт

Уламжлалт хандлага

```
CREATE DATABASE university
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

PostgreSQL сонголтуудтай

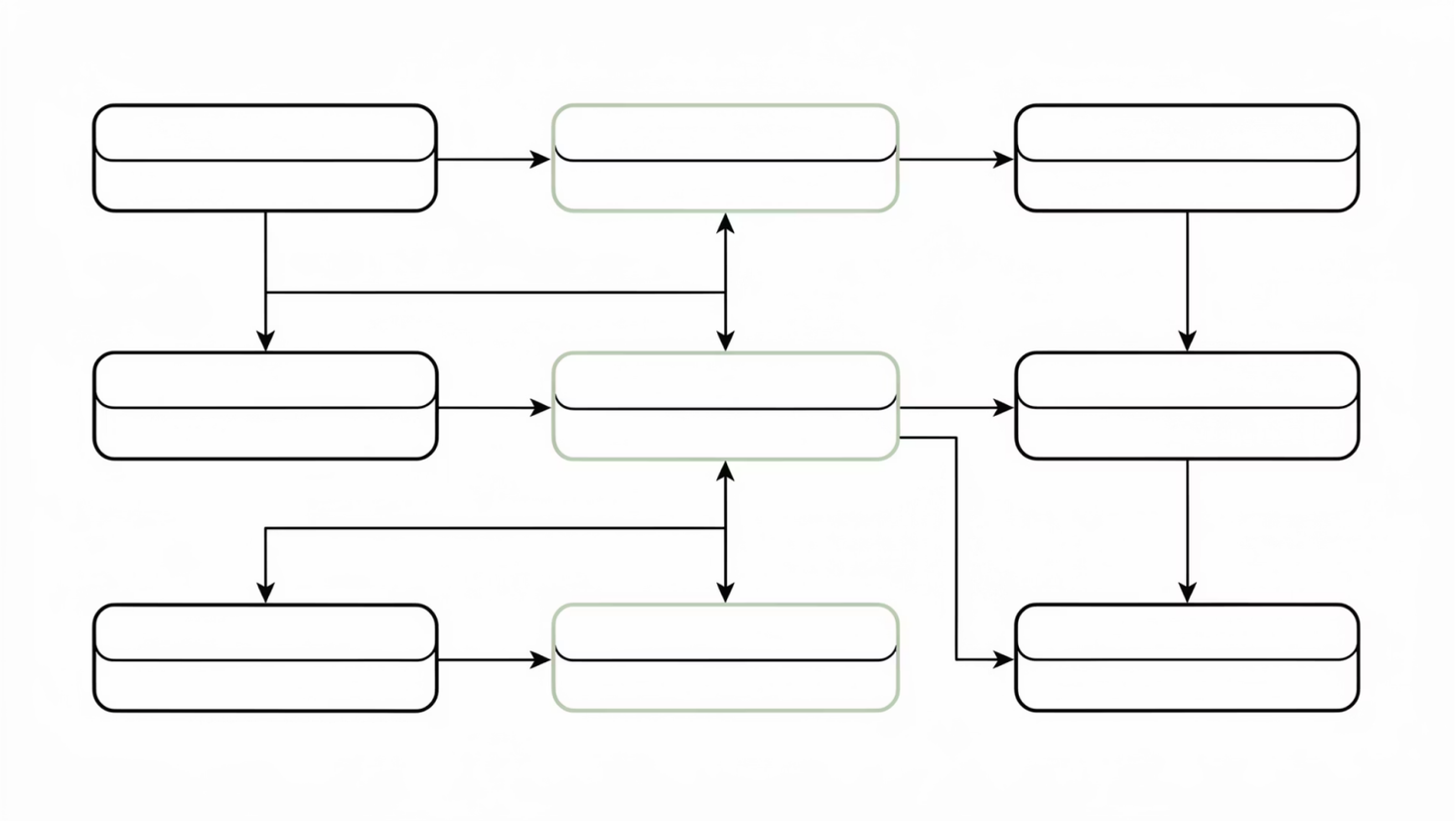
```
CREATE DATABASE university
WITH OWNER = postgres
     ENCODING = 'UTF8'
     LC_COLLATE = 'en_GB.UTF-8'
     LC_CTYPE = 'en_GB.UTF-8';
```

Орчин үеийн үүлэн хандлага

```
-- Google Cloud SQL
CREATE DATABASE university
DEFAULT CHARACTER SET utf8mb4;
```

AWS RDS болон Azure SQL гэх мэт үүлэн платформууд өгөгдлийн сан үүсгэх үйл явцыг ихэвчлэн вэб консол эсвэл infrastructure-as-code хэрэгслүүдээр гүйцэтгэдэг бөгөөд тэмдэгтийн багцын нарийн ширийнийг хийсвэрчилдэг. Гэсэн хэдий ч бодит схемийн тодорхойлолтонд SQL нийцтэй байдлыг хадгалдаг.

CREATE TABLE: Үндсэн бүтэц



Энгийн хүснэгт үүсгэх

```
CREATE TABLE student (
  student_id INT,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  age INT,
  email VARCHAR(100)
);
```

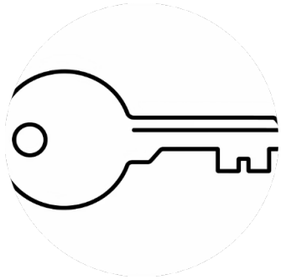
Энэхүү үндсэн синтакс хүснэгт үүсгэдэг боловч хязгаарлалтгүй. Энийг ашиглахгүй бол буруу өгөгдөл, давхардал, бүрэн бүтэн байдлын зөрчлөөс хамгаалагдахгүй.

CREATE TABLE: Хязгаарлалтуудтай (Зөвлөмж)

```
CREATE TABLE student (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  registration_number CHAR(10) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT CHECK (age >= 16 AND age <= 100),
  enrolment_date DATE DEFAULT CURRENT_DATE,
  status VARCHAR(20) DEFAULT 'active',
  department_id INT,
  FOREIGN KEY (department_id)
    REFERENCES department(department_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

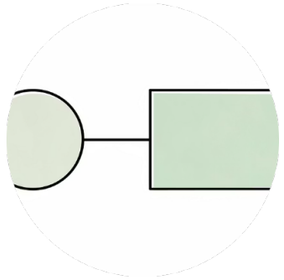
Хязгаарлалтууд нь бизнесийн дүрмийг өгөгдлийн сангийн түвшинд хэрэгжүүлж, буруу өгөгдөл таны системд орохоос өмнө сэргийлдэг. Энэ бол өгөгдлийн бүрэн бүтэн байдлын таны анхны хамгаалалтын шугам юм.

Хязгаарлалтыг ойлгох



PRIMARY KEY

Өвөрмөц таниулагч; NULL эсвэл давхардсан байж болохгүй. Хүснэгт бүр нэг байх ёстой.



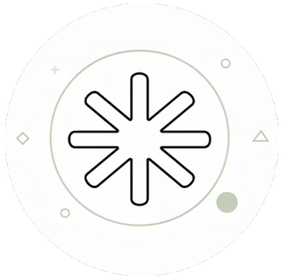
FOREIGN KEY

Өөр хүснэгтийн анхдагч түлхүүрт хандаж, хүснэгтүүдийн хоорондын лавлагааны бүрэн бүтэн байдлыг хангана.



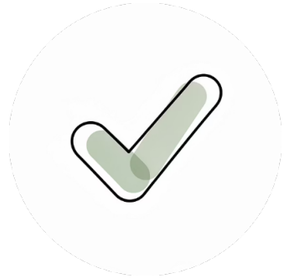
UNIQUE

Давхардлаас сэргийлдэг боловч PRIMARY KEY-ээс ялгаатай нь NULL утгыг зөвшөөрдөг.



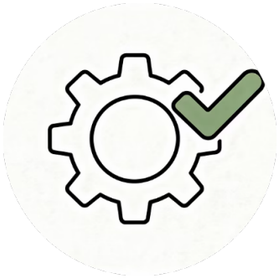
NOT NULL

Талбар үргэлж утга агуулна; хоосон бичлэгийг сэргийлнэ.



CHECK

Өгөгдлийг оруулалт эсвэл шинэчлэл хийхийг зөвшөөрөхөөс өмнө нөхцөлд тохирч байгаа эсэхийг шалгана.



DEFAULT

Оруулах үед ямар ч утга зааж өгөөгүй үед автомат утга өгнө.

Орчин үеийн өргөтгөлүүд: PostgreSQL дэх JSON дэмжлэг

```
CREATE TABLE student (
  student_id SERIAL PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  additional_info JSONB,
  phone_numbers TEXT[],
  profile_text TSVECTOR
);

-- JSON өгөгдлөөр оруулах
INSERT INTO student
  (first_name, additional_info)
VALUES (
  'James',
  '{"hobbies": ["football", "chess"],
    "gpa": 3.8}'::jsonb
);

-- JSON талбарт асуулга тавих
SELECT first_name
FROM student
WHERE additional_info->>'gpa' > '3.5';
```

Уян хатан байдал бүтэцтэй уулзах

- PostgreSQL-ийн JSONB дэмжлэг нь бүтэцтэй өгөгдлийн санд схемийн уян хатан байдлыг өгдөг. Хүснэгтүүдийг дахин загварчлахгүйгээр таамаглах боломжгүй эсвэл хөгжиж буй өгөгдлийн бүтцийг хадгалах боломжтой, гэхдээ ACID шинж чанар болон SQL асуулгын хүчийг хадгалсан хэвээр.

- Энэхүү эрлийз хандлага хоёр ертөнцийн хамгийн сайн нийцлийг санал болгодог: үндсэн өгөгдөлд харилцааны бүрэн бүтэн байдал, нэмэлт мэдээлэлд уян хатан JSON.

Уламжлалт болон орчин үеийн схем загвар

Уламжлалт: Хатуу схем

```
CREATE TABLE student (
  student_id INT PRIMARY
KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  phone_1 VARCHAR(20),
  phone_2 VARCHAR(20),
  phone_3 VARCHAR(20)
);
```

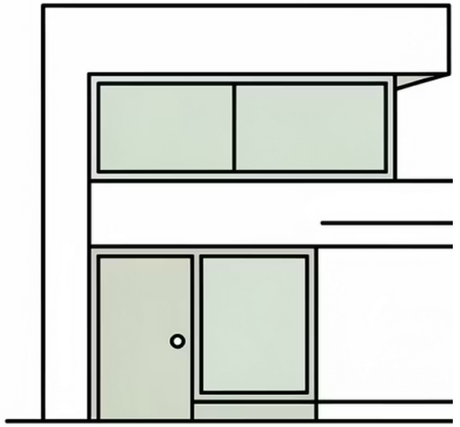
Яг гурван утасны дугаараар хязгаарлагдсан. Илүү нэмэхэд схем өөрчлөлт шаардлагатай.

Орчин үе: Уян хатан схем

```
CREATE TABLE student (
  student_id INT PRIMARY
KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  phone_numbers JSONB
);
```

JSON массив болгон хадгалагдсан хязгааргүй утасны дугаарууд. Схем өөрчлөлт шаардлагагүй.





CREATE INDEX: Асуулга хурдасгах

Индекс гэж юу вэ?

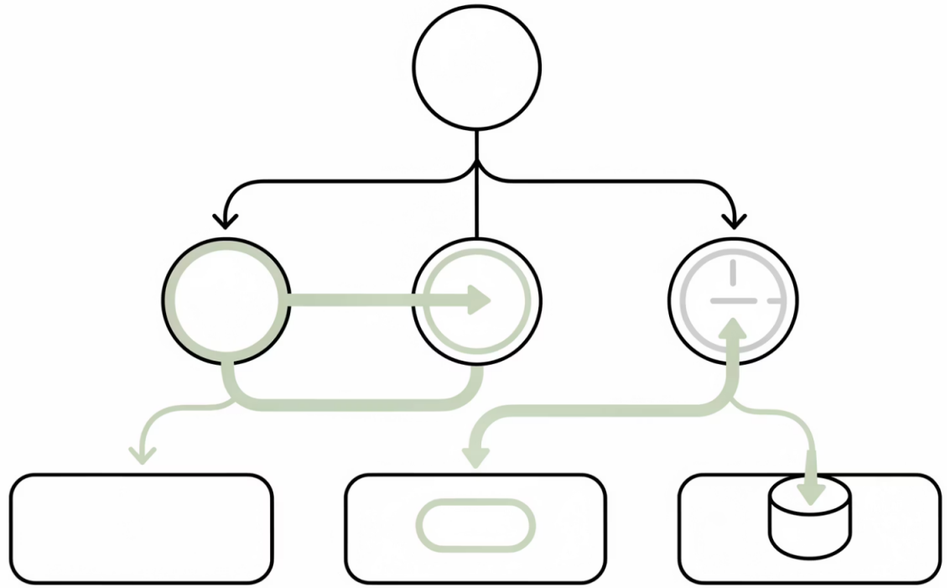
Индексүүдийг номын агуулгын хүснэгт гэж бодоорой — тэд мөр бүрийг уншихгүйгээр мэдээллийг хурдан олоход тусалдаг. Индексгүйгээр өгөгдлийн сан бүх мөрийг сканнердах (хүснэгт скан) ёстой. Индекстэй бол холбогдох бичлэгүүд рүү шууд үсэрнэ.

```
-- Индексгүй: удаан (бүх мөрийг сканнердах)

SELECT * FROM student
WHERE email = 'james@email.com';
-- Гүйцэтгэл: 1М мөрөнд 5000ms

-- Индекс үүсгэх
CREATE INDEX idx_email
 ON student(emaiI);

-- Индекстэй: хурдан (шууд хайлт)
SELECT * FROM student
WHERE email = 'james@email.com';
-- Гүйцэтгэл: 5ms
```



Индексийн төрлүүд

Энгийн индекс

```
CREATE INDEX idx_name
  ON
student(first_name);
```

Нэг баганад асуулгыг хурдасгана

Өвөрмөц индекс

```
CREATE UNIQUE INDEX
idx_reg
  ON
student(registration_
number);
```

Unique **байдлыг хангахын зэрэгцээ асуулгын хурдыг сайжруулна**

Нийлмэл индекс

```
CREATE INDEX
idx_name_surname
  ON
student(first_name,
last_name);
```

Хосолсон асуулгад олон баганыг хамтад нь индексэлнэ

Хэсэгчилсэн индекс

```
CREATE INDEX
idx_active_students
 ON
student(first_name)
 WHERE status =
'active';
```

Нөхцөлд тохирох мөрүүдийг л индексэлнэ (PostgreSQL)

Индексийн буулт–өсөлт: Давуу болон зардал талууд

Давуу талууд

- **SELECT асуулга 100-1000 дахин хурдан**
- **WHERE, JOIN, ORDER BY үйлдлүүд хурдасна**
- **Том өгөгдлийн багцад (1M+ мөр) чухал**
- **Үр дүнтэй эрэмблэх, бүлэглэх боломжтой болгоно**

Сул талууд

- **INSERT, UPDATE, DELETE удаашрана (индекс шинэчлэх шаардлагатай)**
- **Диск зайг их хэмжээгээр эзэлнэ**
- **Хэт олон индекс нийт гүйцэтгэлийг муутгана**
- **Арчлалт, хяналт шаарддаг**

- *Гол нь тэнцвэр юм: WHERE, JOIN болон ORDER BY-д байнга ашигладаг багануудыг индексэл, гэхдээ ховор асуулга тавигддаг багана эсвэл жижиг хүснэгтүүдэд хэт их индекслэхээс зайлсхий.*

ALTER TABLE: Бүтцийг өөрчлөх

Түгээмэл өөрчлөлтүүд

```
-- Багана нэмэх
ALTER TABLE student
ADD COLUMN phone VARCHAR(20);

-- Олон багана нэмэх
ALTER TABLE student
ADD COLUMN address TEXT,
ADD COLUMN birth_date DATE;

-- Багана төрөл өөрчлөх
ALTER TABLE student
MODIFY COLUMN first_name VARCHAR(100);

-- Баганыг дахин нэрлэх
ALTER TABLE student
RENAME COLUMN name TO first_name;

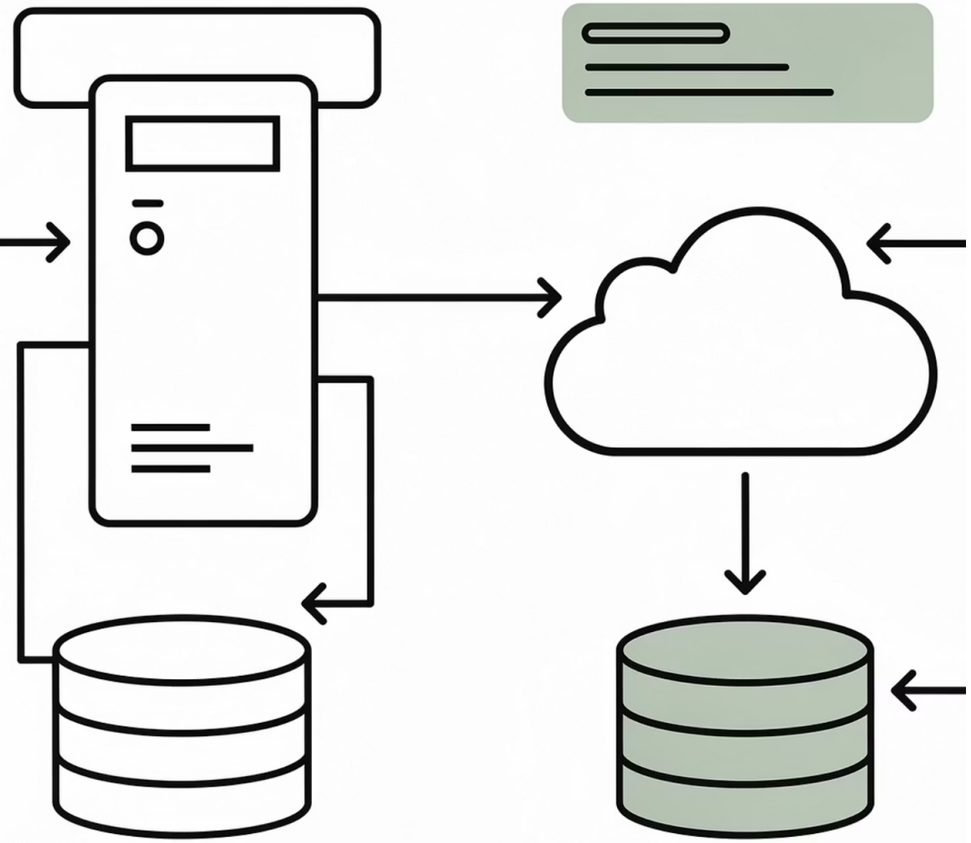
-- Хязгаарлалт нэмэх
ALTER TABLE student
ADD CONSTRAINT chk_age
  CHECK (age >= 16);

-- Хязгаарлалт устгах
ALTER TABLE student
DROP CONSTRAINT chk_age;
```

Хэрэгжүүлэлтэнд анхаарах зүйлүүд:

- Том хүснэгт дээрх ALTER TABLE үйлдлүүд бүхэл хүснэгтийг урт хугацаагаар түгжиж, бусад асуулгыг блоклож болно. 10 сая мөртэй хүснэгт дээр багана нэмэх нь 30 минут зарцуулж болно.

Шилдэг практик: *Хувилбарын хяналттай миграцийн скрипт ашигл.* ***Staging*** *орчинд туршиж үз. Түр зогсолтгүй өөрчлөлтийн хувьд онлайн схем өөрчлөх хэрэгслүүдийг авч үз.*



Уламжлалт болон орчин үеийн схем өөрчлөлтүүд



Уламжлалт хандлага нь хүснэгтийг шууд өөрчилж, үйлчилгээ тасалдуулдаг бол орчин үеийн миграцийн хэрэгслүүд нь хувилбарчлагдсан, эргүүлэх боломжтой, туршигдсан өөрчлөлтүүдийг түр зогсолтгүй хийх боломжийг олгодог.

DROP болон TRUNCATE: Өгөгдөл, бүтэц устгах

DROP: Үүрд устгах

```
-- Бүхэл хүснэгт устгах
DROP TABLE student;

-- Аюулгүй хувилбар (байхгүй бол алдаагүй)
DROP TABLE IF EXISTS student;

-- Өгөгдлийн сан устгах
DROP DATABASE university;

-- Индекс устгах
DROP INDEX idx_email ON student;
```

Анхааруулга: DROP нь буцаагдашгүй! Гүйцэтгэхийн өмнө үргэлж нөөцлөөрэй.

TRUNCATE: Өгөгдөл цэвэрлэх, бүтэц хадгалах

```
--Бүх мөрийг устгах, хүснэгтийн бүтцийг хадгалах
TRUNCATE TABLE student;
```

Шинж чанар	DELETE	TRUNCATE
WHERE нөхцөл	Тийм	Үгүй (бүх мөр)
Хурд	Удаан	Илүү хурдан
Rollback	Боломжтой	Ихэвчлэн үгүй
Triggers	Гүйцэтгэнэ	Гүйцэтгэхгүй
Авто-нэмэгдэл	Үргэлжилнэ	Дахин тохируулна

Практик DDL жишээ: *Их сургуулийн систем*

```sql
CREATE DATABASE university CHARACTER SET utf8mb4; -- Өгөгдлийн сан үүсгэх
USE university;

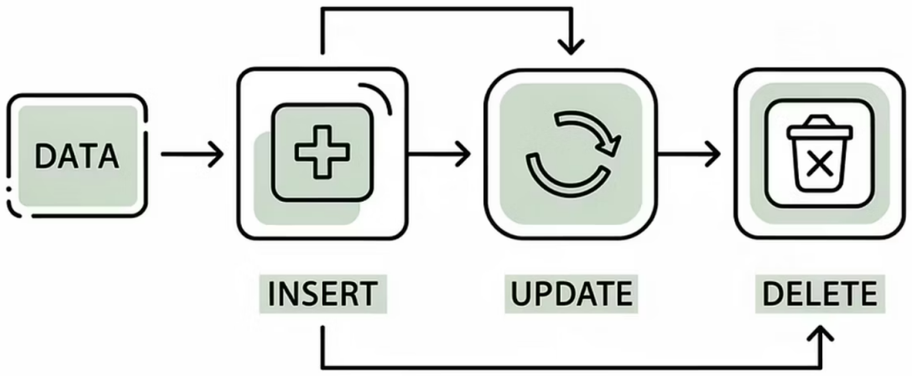
-- Тэнхимүүдийн хүснэгт
CREATE TABLE department (
 department_id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(100) NOT NULL UNIQUE,
 established_year INT,
 head_of_department VARCHAR(100)
);

-- Оюутнуудын хүснэгт хязгаарлалтуудтай
CREATE TABLE student (
 student_id INT PRIMARY KEY AUTO_INCREMENT,
 first_name VARCHAR(50) NOT NULL,
 last_name VARCHAR(50) NOT NULL,
 registration_number CHAR(10) UNIQUE NOT NULL,
 email VARCHAR(100) UNIQUE,
 phone VARCHAR(20),
 address TEXT,
 birth_date DATE,
 age INT CHECK (age >= 16 AND age <= 100),
 department_id INT,
 enrolment_date DATE DEFAULT CURRENT_DATE,
 status ENUM('active', 'suspended', 'graduated') DEFAULT 'active',
 FOREIGN KEY (department_id) REFERENCES department(department_id)
 ON DELETE SET NULL ON UPDATE CASCADE
);

-- Гүйцэтгэлийн индексүүд
CREATE INDEX idx_student_dept
ON student(department_id);

CREATE INDEX idx_student_status
ON student(status);

CREATE FULLTEXT INDEX idx_student_name
ON student(first_name, last_name);
```



БҮЛЭГ 4

DML: Өгөгдөл боловсруулах хэл

DML командууд нь өдөр тутмын өгөгдлийн сангийн үйлдлүүдийн ажлын морь юм. Тэд таньд өгөгдөл үүсгэх, унших, шинэчлэх, устгах боломжийг олгодог — бүх програмыг ажиллуулдаг үндсэн CRUD үйлдлүүд.

INSERT: Хүснэгтэд өгөгдөл нэмэх

Үндсэн синтаксын хувилбарууд

```
-- Бүх баганыг зааж өгөх (зөвлөмжгүй)
INSERT INTO student VALUES
(1, 'James', 'Smith', 'REG123',
 'james@email.com', '555-0100',
 'London', '2000-05-15', 24, 1,
 '2020-09-01', 'active');

-- Сонгосон багануудыг зааж өгөх (зөвлөмж)
INSERT INTO student
  (first_name, last_name,
   registration_number, email,
   department_id)
VALUES
  ('Emma', 'Johnson', 'REG456',
   'emma@email.com', 2);

-- Олон мөрийг нэгэн зэрэг (үр дүнтэй)
INSERT INTO student
  (first_name, last_name,
   registration_number, department_id)
VALUES
  ('Oliver', 'Williams', 'REG789', 1),
  ('Sophia', 'Brown', 'REG012', 2),
  ('Noah', 'Davis', 'REG345', 1);
```

Шилдэг практикууд

- ***Баганы нэрсийг тодорхой зааж өг — энэ нь хүснэгтийн бүтцийн өөрчлөлтөөс хамгаална. Хэн нэгэн багана нэмбэл таны INSERT мэдэгдлүүд эвдрэхгүй.***

Багц оруулга: 1000 мөрийг тус тусд нь оруулах нь ~1000ms шаардана. Нэг мэдэгдэлд оруулах нь ~50ms — 20 дахин сайжруулалт.

```
-- Өөр хүснэгтээс оруулах
INSERT INTO student_2024
 (first_name, last_name, email)
SELECT first_name, last_name, email
FROM student
WHERE enrolment_date >= '2024-01-01';
```

Орчин үеийн INSERT өргөтгөлүүд

UPSERT (PostgreSQL)

```
INSERT INTO student
  (student_id, first_name, email)
VALUES (1, 'James', 'james_new@email.com')
ON CONFLICT (student_id)
DO UPDATE SET email = EXCLUDED.email;
```

Бичлэг байвал шинэчилж, байхгүй бол оруул — уралдааны нөхцөлийг арилгана

RETURNING заалт

```
INSERT INTO student
  (first_name, last_name,
   registration_number)
VALUES ('James', 'Smith', 'REG123')
RETURNING student_id, first_name;
```

Хоёр дахь асуулгагүйгээр шинээр оруулсан ID болон утгуудыг ав

SELECT: Өгөгдөл сэргээн авах үндэс

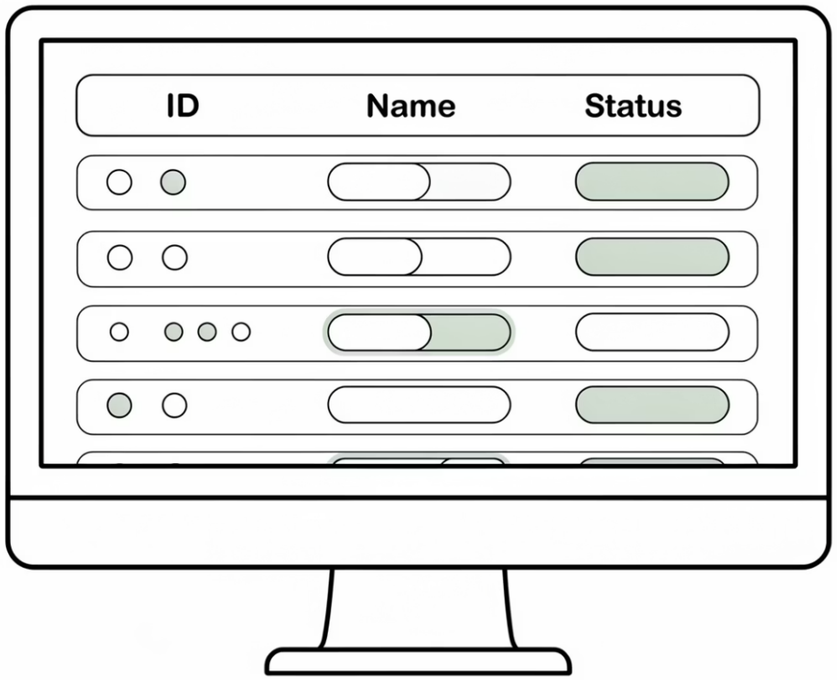
Үндсэн сонголт

```
-- Бүх багана (үйлдвэрлэлд зайлсхий)
SELECT * FROM student;

-- Тодорхой баганууды (зөвлөмж)
SELECT student_id, first_name,
       last_name, email
FROM student;

-- Давхардал арилгах
SELECT DISTINCT department_id
FROM student;

-- Тооцоолсон баганууды
SELECT first_name, last_name,
  CONCAT(first_name, ' ', last_name)
    AS full_name,
  YEAR(CURRENT_DATE) - YEAR(birth_date)
    AS current_age
FROM student;
```



WHERE заалт: Өгөгдөл шүүх

Харьцуулах үйлдлүүд

```
WHERE age > 20
WHERE age >= 18
WHERE age < 25
WHERE status !=
'graduated'
WHERE status <>
'graduated'
```

BETWEEN & IN

```
WHERE age
BETWEEN 18 AND
25
WHERE
department_id IN
(1, 2, 3)
WHERE status IN
  ('active',
'suspended')
```

Загвараар таарах

```
WHERE first_name
LIKE 'J%'
-- J-ээр эхэлнэ
WHERE first_name
LIKE '%son'
-- son-оор төгсөнө
WHERE email LIKE
'%@gmail.com'
-- Gmail хаягууд
```

NULL боловсруулалт

```
WHERE email IS NULL
WHERE email IS NOT
NULL
-- БУРУУ:
-- WHERE email =
NULL
-- ЗӨВӨӨР:
-- WHERE email IS
NULL
```

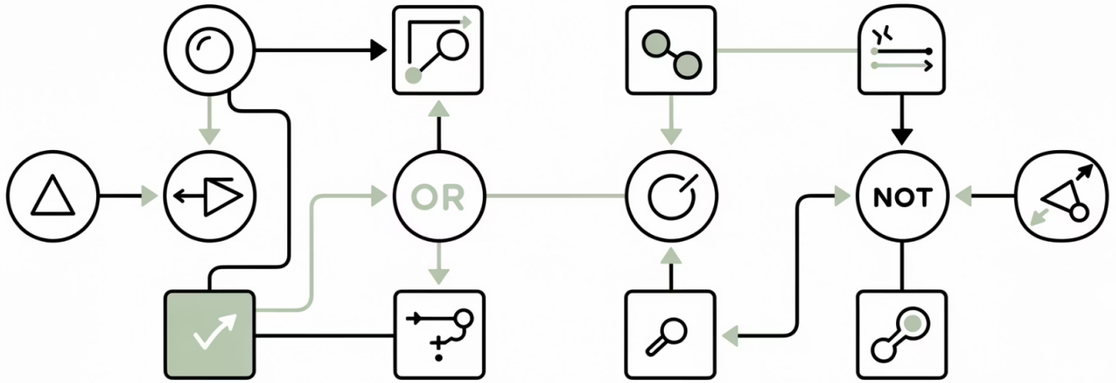
Логик үйлдлүүд болон нарийн нөхцөлүүд

```sql
-- AND: бүх нөхцөл үнэн байх ёстой
SELECT * FROM student
WHERE department_id = 1
  AND age > 20
  AND status = 'active';

-- OR: ямар ч нөхцөл үнэн байж болно
SELECT * FROM student
WHERE department_id = 1
  OR department_id = 2;

-- NOT: нөхцөл үгүйсгэх
SELECT * FROM student
WHERE NOT (status = 'graduated');

-- Хаалттай нарийн хослолууд
SELECT * FROM student
WHERE (department_id = 1 OR department_id = 2)
  AND age > 20
  AND (status = 'active' OR status =
'suspended');
```



*Хаалт нь үнэлэлтийн дарааллыг хянадаг. Тэдгүйгээр **AND** нь **OR**-оос өндөр давуу эрхтэй бөгөөд энэ нь санаанд оромгүй үр дүнг гаргаж болно. Тодорхой байдлын тулд үргэлж хаалт ашигла.*